

---

## CS5787: Exercises 1

Python Notebook and README Practical Section [here](#).

Avinash Nair  
an586

Jasper Levy  
jl4537

### 1 Theory: Question 1 [12.5 pts]

- A. The shape of the input  $X$  is  $(m, 10)$ .
- B. The shape of the hidden layer's weight vector ( $W_h$ ) is  $(10, 50)$  and the bias vector ( $b_h$ ) is  $(50, )$ .
- C. The shape of the output layer's weight vector ( $W_o$ ) is  $(50, 3)$  and the bias vector ( $b_o$ ) is  $(3, )$ .
- D. The shape of the output matrix  $Y$  is  $(m, 3)$ .
- E.  $Y = \text{RELU}(W_h + b_h) W_o + b_o$

### 2 Theory: Question 2 [12.5 pts]

In the first layer, we have a  $3 \times 3$  kernel with 3 channels for an RGB image. As such, each weight filter has dimension  $3 \times 3 \times 3$ . The first layer outputs 100 feature maps and each filter has 1 bias, so the parameters for the first layer is equal to  $3 \times 3 \times 3 \times 100 + 100 = 2,800$  parameters. The second layer has the 100 feature maps from the previous layer as input and 200 feature maps (each with one bias) as output. The kernel is still  $3 \times 3$ , so the parameters for the second layer are equal to  $3 \times 3 \times 100 \times 200 + 200 = 180,200$ . The third layer has the previous layer's 200 feature maps as input, and 400 feature maps with their individual biases as output. Each filter is of shape  $3 \times 3 \times 200$ , so the number of parameters for this layer is  $3 \times 3 \times 200 \times 400 + 400 = 720,400$ . Adding each layer's parameters together yields a total of 903,400 parameters for the entire CNN.

### 3 Theory: Question 3 [25 pts]

a.

$$\frac{\partial f}{\partial \gamma} = \sum_{i=1}^m \frac{\partial f}{\partial y_i} \cdot \hat{x}_i$$

b.

$$\frac{\partial f}{\partial \beta} = \sum_{i=1}^m \frac{\partial f}{\partial y_i}$$

c.

$$\frac{\partial f}{\partial \hat{x}_i} = \frac{\partial f}{\partial y_i} \cdot \gamma$$

d.

$$\frac{\partial f}{\partial \sigma^2} = \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot (x_i - \mu) \cdot -\frac{1}{2} \cdot (\sigma^2 + \epsilon)^{-\frac{3}{2}}$$

---

e.

$$\frac{\partial f}{\partial \mu} = \sum_{i=1}^m \frac{\partial f}{\partial \hat{x}_i} \cdot -\frac{1}{\sqrt{\sigma^2 + \epsilon}}$$

f.

$$\frac{\partial f}{\partial x_i} = \frac{\partial f}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma^2 + \epsilon}} + \frac{\partial f}{\partial \sigma^2} \cdot \frac{2(x_i - \mu)}{m} + \left( \sum_{j=1}^m \left( \frac{\partial f}{\partial \hat{x}_j} \cdot -\frac{1}{\sqrt{\sigma^2 + \epsilon}} \right) \right) \cdot \frac{1}{m}$$

## 4 Practical [50 pts]

See code and README [here](#).

### Architecture Adjustment

The FashionMNIST dataset consists of images that are of dimensions 28x28. Seeing that the LeNet5 architecture expects images of size 32x32, we simply padded the images from the FashionMNIST dataset to work with the LeNet5 model. We also normalized the images using torchvision transforms. Upon preparing the dataset to fit our models, we then set up data loaders to enable mini-batch sampling.

For deciding dropout percentages, we initially chose  $p=0.5$  and then decided to tune  $p$  as a hyperparameter. We started with  $p=0.5$  because research on preventing overfitting with dropout suggested, "p can be chosen using a validation set or can simply be set at 0.5, which seems to be close to optimal for a wide range of networks and tasks" (Srivastava et al., 2014) - [Srivastava et al., 2014](#).

### Conclusion & Discussion

Each technique yielded test accuracies between 89% and 90% after the model with optimal hyperparameters was trained. The model with the highest accuracy on unseen test data was the dropout model with the following hyperparameters:

- **batch\_size:** 32
- **learning\_rate:** 0.001
- **optimizer:** Adam
- **dropout\_rate:** 0.2
- **weight\_decay:** 0.0

Interestingly, we find the best model has the lowest of all of our included dropout rates (0.2). The dropout regularization technique works by decreasing reliance on specific weights in training so that the model can generalize better. We see here that this model generalizes the best when encountering new data.

Another notable result is that the non-regularized model (the base model) had the lowest of all test accuracies. This is to be expected, for without regularization, the model will not generalize to test data as well.

Though the differences among the regularization techniques are small, these differences could be significant in high-stakes image classification tasks. More importantly, what we don't see here is the use of multiple regularization techniques together. Overall, this analysis shows the benefits of regularization through the results. Regularization should certainly be used to maximize test accuracy when using Convolutional Neural Networks for image classification.

---

Please see [code and README](#) for more info on how to train and test with saved weights as well as the accuracy vs. epoch plots for each regularization technique. The README and python notebook will also have a table to summarize the quantitative findings.

The linked folder contains the README markdown file, a copy of the README in a google doc (to see visualizations and avoid compiling), and the .ipynb notebook itself.